

UNIVERSIDAD DE GUANAJUATO

División de Ingenierías del Campus Irapuato-Salamanca (DICIS)

Bases de Datos NO Relacionales

Proyecto Final

Jorge Antonio Barroso Solorzano

María Isabel Rocha Vargas

José Adrián Rodríguez González

Hector Emilio Valdes Velazquez

4 de diciembre de 2025

Introducción

El presente proyecto final, desarrollado para la Unidad de Aprendizaje de "Bases de Datos No Relacionales", tiene como objetivo principal el diseño e implementación de un sistema de gestión de noticias robusto, escalable y eficiente, denominado "Portal News" (inspirado en el ficticio "Daily Bugle"). En el contexto actual del desarrollo web, las aplicaciones modernas demandan tiempos de respuesta inmediatos y una alta disponibilidad, requisitos que a menudo exceden las capacidades de un modelo de base de datos relacional único.

El sistema se fundamenta en la orquestación de tres motores NoSQL líderes: MongoDB para la gestión flexible de contenido no estructurado (noticias), Apache Cassandra para el manejo masivo de registros de auditoría y usuarios con alta disponibilidad, y Redis como almacén en memoria para optimizar la seguridad y gestión de sesiones mediante caché.

El desarrollo abarca una solución Full Stack contenerizada mediante Docker, integrando un Frontend reactivo construido con SvelteKit y un Backend modular en Node.js/Express. A lo largo de este reporte, se detallará el diseño de la arquitectura distribuida, la justificación técnica de cada base de datos seleccionada, los flujos de seguridad implementados (autenticación y autorización) y se presentarán las pruebas de rendimiento que validan la eficiencia del sistema, demostrando tiempos de respuesta óptimos para un entorno de producción real.

Desarrollo

Definición de lógica de negocio

1 – Diseño, Justificación y Arquitectura de Datos:

Para el desarrollo del Portal de Noticias, se optó por una arquitectura que permita utilizar tecnologías de almacenamiento especializadas que se adaptan mejor a la naturaleza de cada tipo de dato, sin forzar los requerimientos en un único sistema. Se seleccionaron las siguientes bases de datos para la realización de los objetivos:

1. MongoDB (Gestor de contenido/ Noticias)

Se decantó por MongoDB como base de datos documental para el almacenamiento del núcleo del negocio de noticias.

MongoDB tiene una naturaleza hecha para los Datos (Schema-less) y es aprovechado de manera puntual debido a lo volátil que es el contenido periodístico. Una noticia puede contener estructuras bastante complejas generadas por un editor de texto (TipTap JSON), múltiples metadatos, arreglos de etiquetas o enlaces multimedia. Un módelo SQL sería demasiado rígido y requeriría migraciones de esquemas computacionalmente costosas ante cambios en el formato de la editorial, mientras que MongoDB permite almacenar estos documentos con flexibilidad total.

De manera simultánea, se tiene que tomar en cuenta que, en un portal de noticias, la proporción de lecturas frente a estructuras es muy alta (muchos visitantes, pocos reporteros). MongoDB ofrece un rendimiento bastante bueno frente escenarios de lectura y permiten, por ejemplo, recuperar un artículo completo (titulo, cuerpo, autor…) con una sola operación sin la necesidad de realizar JOINs.

Específicamente, para la estructura interna de los documentos, se decantó por implementar una estrategia de desnormalización controlada. En lugar de almacenar únicamente el ID del autor, también su nombre estará dentro del documento de la noticia. Esta decisión arquitectónica elimina la latencia de red y la complejidad que implicaría realizar una "unión de aplicación" con la base de datos de usuarios (Cassandra) cada vez que se renderiza el feed principal. Asimismo, se define el campo de contenido como un objeto complejo (Object) en lugar de texto plano para persistir intacta la estructura jerárquica JSON generada por el editor TipTap, y añadimos un campo slug único e indexado para garantizar URLs amigables (SEO), evitando exponer los identificadores hexadecimales internos de la base de datos en la navegación del usuario.

2. Apache Cassandra (Usuarios y Auditoría)
   Se seleccionó Cassandra como base de datos distribuida para gestionar la información de usuarios y, críticamente, la bitácora de auditoria (logger).
   Cassandra cuenta con escalabilidad y una estructura masiva (Bitácora). El sistema de logs requiere de una capacidad de escritura intensiva y constante (“append-only”). Cassandra está diseñada para soportar volúmenes masivos de escritura con latencia mínima, lo cual es idea para registrar cada acción realizada en el sistema (creación, edición, registro) sin degradar el rendimiento de la aplicación principal, lo anterior se puede ver reflejado en la gestión de usuarios, donde se prioriza la disponibilidad y la tolerancia a particiones (AP). Cassandra asegura que el servicio de autenticación y

consulta de perfiles permanezca operando incluso en caso de fallos en nodos del clúster, garantizando la robustez del sistema en un entorno de nube distribuido.

La estructura de datos se ha diseñado siguiendo el principio de Querry-Driven Desing, optimizando las tablas para responder preguntas especificas (p.e. “¿Qué acciones realizó el usuario X en tal fecha?”) de manera instantánea. El diseño de tablas en Cassandra se aleja de la normalización relacional tradicional. Para la tabla de bitácora, se definió una Clave Primaria Compuesta particionada por ID de usuario y ordenada por fecha descendente; esto agrupa físicamente todas las acciones de un mismo usuario en el disco y las ordena cronológicamente, permitiendo lecturas secuenciales instantáneas del historial de actividad sin necesidad de ordenamiento en memoria. Por otro lado, en la tabla de usuarios, implementamos un índice secundario sobre el campo "email", lo cual es indispensable para habilitar la búsqueda de usuarios por correo durante el proceso de inicio de sesión, ya que Cassandra por defecto limita las búsquedas únicamente a la clave primaria.

3. Redis (Sesiones y Caché)
   Redis se implementa como almacén de estructura clave-valor en memoria (In-Memory) para la gestión de sesiones y seguridad.

   La verificación de autenticación de Redis (Middleware de seguridad) se ejecuta en cada petición HTTP que recibe el servidor. Realizar esta validación contra una base de datos en disco (como Mongo o Cassandra) generaría un cuello de botella. Redis, al operar en memoria RAM, ofrece tiempos de respuesta inmediatos (complejidad O(1)), garantizando que la seguridad no comprometa la velocidad de navegación.

   Otra cuestión para tomar en cuenta fue la volatilidad controlada de Redis con las sesiones de usuarios que son efímeras. Redis maneja nativamente el concepto de TTL, permitiendo que los tokens de sesión y las cookies expiren y se autodestruyan automáticamente después de un periodo de inactividad (configurado a 2 horas), simplificando la lógica de limpieza de datos en el backend.

   Para la estructura interna, se descartó el uso de tipos complejos como HashMaps o Listas y optamos por la estructura más primitiva de Redis: Cadenas (Strings) simples bajo el patrón de nombrado token:uuid_usuario en conjunto de un diseño plano porque el Token JWT es un dato atómico e inmutable que no requiere lecturas parciales. Esta decisión arquitectónica garantiza matemáticamente ya mencionada, asegurando que el tiempo de verificación sea constante e inmediato, sin importar si

hay diez usuarios o un millón conectados simultáneamente, manteniendo la memoria utilizada al mínimo indispensable al guardar solo la cadena del token cifrado.

2 – Arquitectura y lógica que intercala los servicios

2.1.- Arquitectura General de Infraestructura

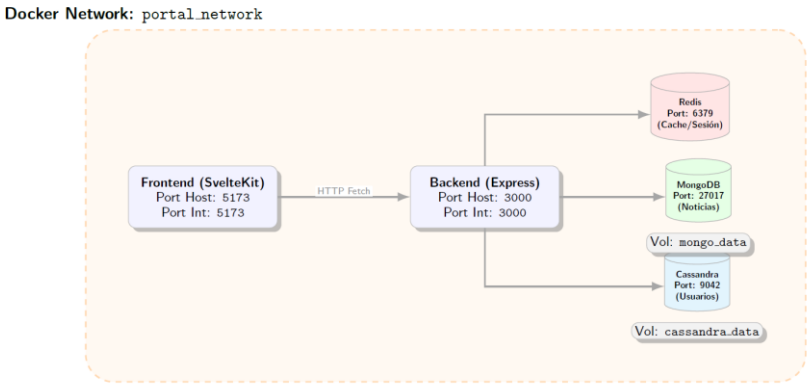
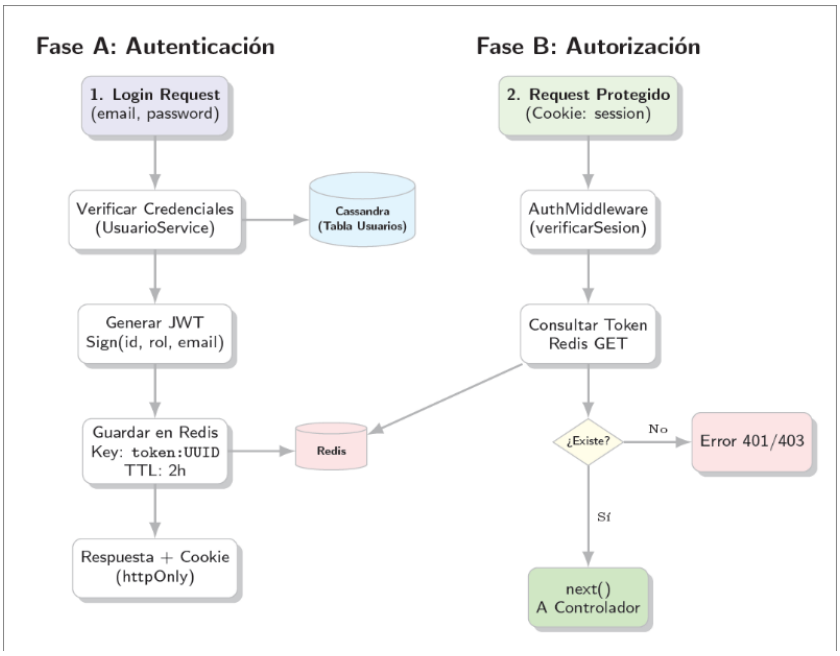


Ilustración 1 - Arquitectura General de Infraestructura

El Diagrama de Arquitectura General ilustra la infraestructura contenerizada del proyecto orquestada mediante Docker Compose, diseñada para ofrecer un entorno robusto y escalable. En el nivel de aplicación, se presentan dos servicios principales: un Frontend desarrollado en SvelteKit expuesto en el puerto 5173 para la interacción del usuario, y un Backend construido con Express y Node.js en el puerto 3000 que actúa como el núcleo de la API. Estos servicios se comunican a través de una red interna de Docker denominada portal_network, aislada del exterior para mayor seguridad. La capa de datos se distribuye estratégicamente en tres motores especializados según la naturaleza de la información: MongoDB se utiliza para el almacenamiento de contenido no estructurado como las noticias, Apache Cassandra gestiona los datos críticos de usuarios y registros de auditoría debido a su alta disponibilidad, y Redis opera como una base de datos en memoria para gestionar sesiones y caché, optimizando la latencia. La persistencia de los datos críticos se garantiza mediante volúmenes de Docker dedicados para Mongo y Cassandra.

2.2.- Ciclo Completo de Seguridad y Autenticación

Ilustración 2- Ciclo Completo de Seguridad y Autenticación



El Diagrama de Flujo de Seguridad detalla la estrategia híbrida implementada para gestionar la identidad y el acceso de los usuarios de manera eficiente. El proceso inicia con la solicitud de inicio de sesión, donde el backend verifica las credenciales recibidas (correo y contraseña) consultando la base de datos Cassandra. Una vez validada la identidad, el sistema genera un JSON Web Token (JWT) firmado que incluye el ID y el rol del usuario, y lo almacena inmediatamente en Redis con un tiempo de vida (TTL) definido de dos horas, devolviendo una cookie segura al cliente. Para las solicitudes subsiguientes a rutas protegidas, un middleware de autenticación intercepta el tráfico y consulta Redis para verificar la validez de la sesión en tiempo real. Este diseño permite que la verificación de sesiones sea extremadamente rápida al realizarse en memoria, evitando sobrecargar Cassandra con lecturas repetitivas y garantizando que la revocación de acceso (logout) sea instantánea al eliminar la clave de Redis.

2-3.- Ciclo de Vida de la Noticia

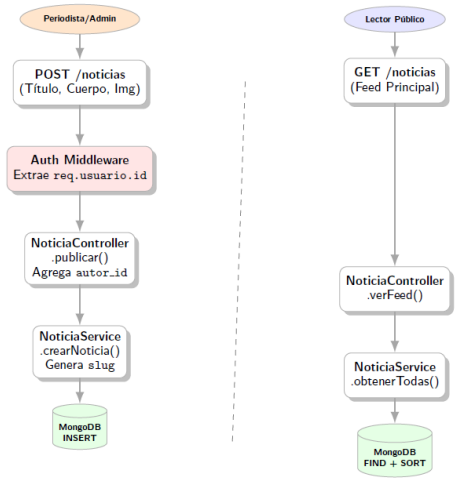


Ilustración 3 - Ciclo de Vida de la Noticia

El Diagrama del Ciclo de Vida de la Noticia mapa el flujo completo de la entidad de negocio principal, diferenciando claramente entre las operaciones de escritura y lectura. En el flujo de escritura, un usuario con rol de Periodista o Administrador envía una solicitud de creación que atraviesa el middleware de seguridad para inyectar automáticamente la identidad del autor validada previamente. Posteriormente, la capa de servicio procesa los datos, generando identificadores amigables para URLs (slugs) antes de insertar el documento final en la colección de MongoDB. Por otro lado, el flujo de lectura permite a los usuarios públicos acceder al contenido mediante solicitudes GET, donde el controlador invoca al servicio para recuperar, ordenar y optimizar los documentos desde MongoDB antes de enviarlos como respuesta JSON. Esta separación lógica asegura que la integridad de los datos se mantenga estricta durante la creación, mientras que la entrega de contenido se optimiza para la velocidad y el consumo masivo.

3 – Velocidad de Consultas y Servicios

El objetivo principal de estas pruebas fue verificar la latencia y la integración de los servicios de backend en un entorno contenerizado con Docker, simulando el flujo de trabajo crítico de un periodista. Para estas mediciones se utilizó la herramienta cURL con medición de tiempo total de respuesta.

En la primera prueba se evaluó el Registro de Usuario. Esta operación consistió en la creación de un nuevo usuario con rol de Journalist. El flujo de datos implicó que el Backend en Nodejs realizara una inserción directa en Apache Cassandra. El resultado de tiempo obtenido fue de 0.073 segundos, es decir, 73 milisegundos. El análisis de este resultado indica que el tiempo de respuesta es óptimo, demostrando la capacidad de Cassandra para manejar escrituras rápidas sin latencia perceptible y confirmando la estabilidad de la conexión interna.

La segunda prueba se centró en la Autenticación o Login. La descripción del proceso fue la validación de credenciales y la generación de una sesión segura. El flujo de datos requirió que el Backend consultara las credenciales en Cassandra mediante una lectura y posteriormente generara un JWT para almacenarlo en Redis con un tiempo de vida definido. El resultado de tiempo fue de 0.063 segundos, equivalentes a 63 milisegundos. A pesar de requerir interacción con dos motores de base de datos distintos en una sola petición, el tiempo se mantuvo por debajo de los 100 milisegundos, validando la eficiencia de usar Redis para la gestión de sesiones volátiles.

La tercera prueba analizó la Publicación de Contenido. Se simuló el envío de un artículo noticioso protegido por autenticación. El flujo de datos involucró que el Middleware verificara la validez del token en Redis mediante una lectura y luego el Backend procesara e insertara el documento JSON en MongoDB. El resultado de tiempo fue de 0.037 segundos, o 37 milisegundos. Este fue el resultado más rápido de todas las pruebas, confirmando que la validación en memoria con Redis es prácticamente instantánea y que MongoDB está altamente optimizado para la ingesta de documentos.

Como conclusión general, la arquitectura propuesta demuestra un rendimiento excepcional, manteniendo todas las operaciones críticas de escritura, lectura y autenticación consistentemente por debajo del umbral de 100 milisegundos. El sistema es altamente reactivo y la segregación de responsabilidades entre MongoDB, Cassandra y Redis justifica la complejidad de la infraestructura al ofrecer tiempos de respuesta mínimos.

Definición de llamadas de API

La comunicación entre la capa de presentación (Frontend en SvelteKit) y la lógica de negocio (Backend en Node.js) se estructuró bajo una arquitectura RESTful, utilizando JSON como formato estándar para el intercambio de datos. El backend expone sus servicios a través del puerto 3000, organizando las rutas en dos módulos principales definidos en el enrutador maestro: /api/usuarios y /api/noticias.

A continuación, se detallan los flujos de comunicación implementados:

- **Gestión de Autenticación y Sesiones**
  El acceso al sistema se gestiona mediante endpoints que interactúan directamente con los servicios de Redis y Cassandra.

  Inicio de Sesión (Login): El cliente realiza una petición POST a /api/usuarios/login enviando las credenciales (email y contraseña). El backend verifica la identidad y, en caso de éxito, establece una cookie httpOnly con el token de sesión. En el frontend, esta llamada se configura con la propiedad credentials: 'include' para permitir la persistencia de la cookie de sesión en el navegador.

  Cierre de Sesión (Logout): Se ejecuta mediante una petición POST a /api/usuarios/logout, la cual instruye al servidor para invalidar el token en Redis y eliminar la cookie del cliente.

- **Módulo de Noticias (Público y Privado)**

Este módulo maneja el núcleo del portal, diferenciando entre operaciones de lectura masiva y escritura privilegiada.

Lectura (Feed): La página principal consume el endpoint GET /api/noticias. Esta llamada es pública y recupera la lista de artículos ordenados cronológicamente desde MongoDB.

Lectura Individual: Para acceder al detalle de una nota, se utiliza GET /api/noticias/:slug, permitiendo la navegación mediante URLs amigables.

Publicación: Los reporteros autenticados envían datos mediante POST a /api/noticias. Esta ruta está protegida por el middleware de autenticación que valida el token antes de permitir la escritura en la base de datos.

- **Módulo de Administración de Usuarios**
  Diseñado exclusivamente para el rol de administrador, este conjunto de endpoints permite la gestión completa del ciclo de vida de los usuarios (CRUD).

  Registro: Se utiliza POST /api/usuarios/registro para crear nuevos perfiles (Reporteros o Admins) en Cassandra.

  Listado y Detalle: El dashboard administrativo consume GET /api/usuarios/ para listar todos los registros y GET /api/usuarios/:id para obtener los datos de un usuario específico antes de una edición.

  Edición y Eliminación: Se implementaron los métodos PUT /api/usuarios/:id para actualizar información y DELETE /api/usuarios/:id para dar de baja usuarios, asegurando que solo el administrador pueda ejecutar estas acciones destructivas.

- **Implementación Técnica en el Cliente**
  Las llamadas a la API se realizan utilizando la función nativa fetch de JavaScript. Se implementó una estrategia híbrida de consumo de datos:

  Client-Side Fetching: Para interacciones dinámicas como el login o el borrado de usuarios, las llamadas se ejecutan directamente desde el navegador en los componentes .svelte.

  Server-Side Loading (SSR): Para la protección de rutas y la carga inicial de datos, se utiliza el archivo +layout.server.js y +page.server.js. Esto permite que el servidor de SvelteKit realice peticiones internas al backend (usando la red de Docker) para

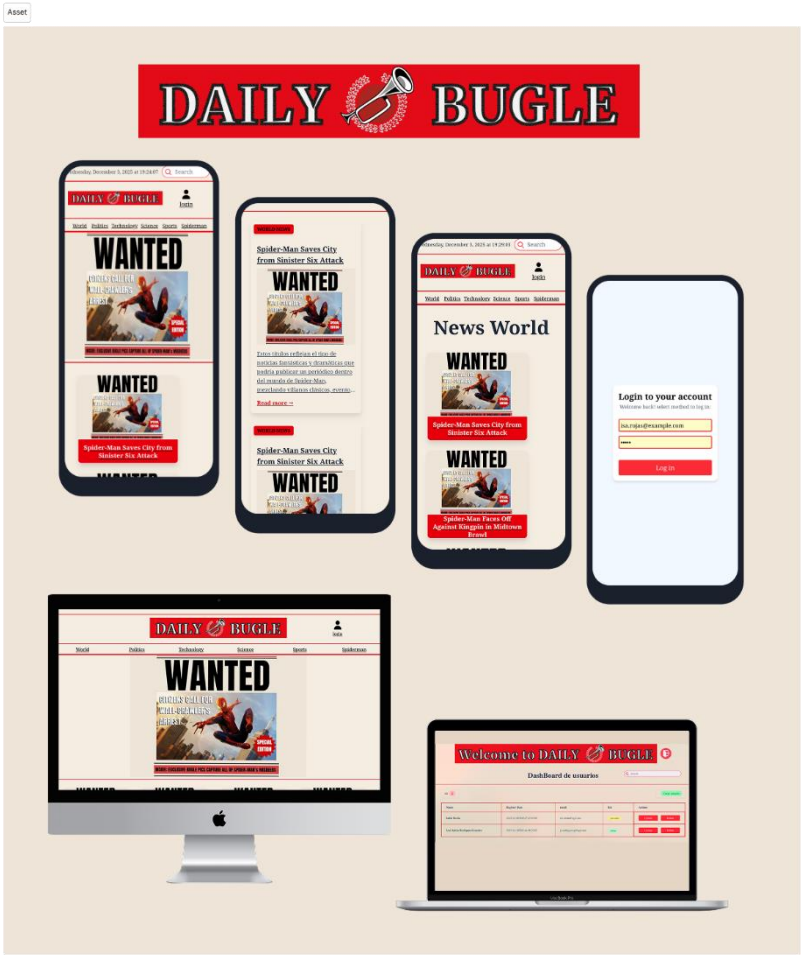
verificar la validez de la sesión antes de renderizar la interfaz de usuario, garantizando una capa extra de seguridad y rendimiento.

Frontend

La primera fase del frontend fue diseñar la página web con figma, esto con el fin de que, al momento de programarlo fuera más sencillo, esta herramienta nos agilizo el diseñar la página web con sus plantillas disponibles de tablas, logos y botones, además para tener un concepto se tomó como inspiración el diario ficticio de Spiderman el “Daily Bugle” o “El clarín” en español, todos los logos fueron diseñados desde cero con esta herramienta.

Una vez finalizado el diseño se empezó a programar, se creó el proyecto con el framework SvelteKit el cual nos da la posibilidad de crear componentes reutilizables. Algunas de sus características son: Elimina el uso de un virtual Dom, permite una estructura organizada, el manejo de rutas es simple, mejora el rendimiento SEO y optimiza los tiempos de carga iniciales. El otro framework con el que se trabajo fue Tailwind un framework de CSS, este nos ayudó al agilizar el manejo de CSS ya que tiene clases predefinidas las cuales nos ayudan a darle un diseño estético y uniforme a los todos los componentes de la página web y además incluye herramientas de optimización como PurgeCSS, que elimina las clases no utilizadas en la producción, resultando en un archivo CSS más ligero y rápido de cargar.

Con estas herramientas se creó un sitio no solo funcional si no también estético para cualquier usuario de computadoras, tabletas y teléfonos utilizando técnicas responsive para un buen diseño y estructura de la pagina web. En las siguientes figuras se observa la página:



Despliegue en la nube

Para la fase de producción, se realizó una migración desde el entorno de desarrollo local (Docker Compose) hacia una arquitectura distribuida en la nube. Durante el análisis de la API, se determinó que un despliegue monolítico en servicios *serverless* (como Vercel para todo el stack) no era viable debido a la naturaleza persistente de las conexiones requeridas por el Backend y la complejidad de los controladores de base de datos. Por consiguiente, se adoptó una estrategia híbrida utilizando **Vercel** y **Railway**.

- Estrategia de Frontend (Vercel)

La capa de presentación, desarrollada en **SvelteKit**, se desplegó en **Vercel**. Esta plataforma fue seleccionada por su optimización nativa para frameworks modernos de JavaScript, permitiendo una entrega de contenido estática y renderizado del lado del servidor (SSR) de alta velocidad. El frontend actúa ahora como el punto de entrada público, consumiendo los servicios del backend a través de peticiones HTTP seguras.

- Estrategia de Backend (Railway)

La lógica de negocio (API REST en Express/Node.js) se migró a **Railway**. Esta decisión fue crítica debido a que el backend requiere un entorno de ejecución persistente para mantener los *pools* de conexión activos con los tres motores de base de datos y gestionar los tiempos de espera de las transacciones complejas, algo que las funciones *serverless* limitan estrictamente.

- **Capa de Persistencia y Datos (Estrategia Híbrida)**

Debido a las limitaciones de servicios simultáneos en el plan gratuito de Railway, se descentralizó la capa de datos aprovechando soluciones especializadas de *Database-as-a-Service* (DBaaS):

1. **Redis (Railway):** Se mantuvo dentro de Railway, junto al backend. Esto garantiza la mínima latencia posible (comunicación interna) para la validación de sesiones en tiempo real, que es la operación más frecuente del sistema.
2. **MongoDB (MongoDB Atlas):** La base de datos documental se migró a **Atlas**, aprovechando su infraestructura gestionada en la nube. Esto asegura la disponibilidad y el respaldo automático de los documentos de noticias sin consumir los "slots" de servicio limitados en Railway.
3. **Apache Cassandra (DataStax Astra DB):** Para la gestión de usuarios y auditoría, se utilizó **Astra DB**, un servicio *cloud-native* de Cassandra gestionado por DataStax. Esto eliminó la complejidad de administrar un clúster de Cassandra manual (que consume muchos recursos de RAM) y permitió conectar el backend mediante *Secure Connect Bundles* (SCB) para una autenticación cifrada y segura.

- Interconexión y Variables de Entorno

Para integrar estos servicios distribuidos, se reconfiguraron las variables de entorno (`ENV`) del sistema. Las referencias a `localhost` y nombres de contenedores internos (`mongo`, `cassandra`, `redis`) fueron reemplazadas por las **URLs privadas y públicas** proporcionadas por Railway. Esto asegura que el Frontend en Vercel pueda comunicarse con

el Backend, y que el Backend tenga acceso seguro y autenticado a la capa de datos, cerrando el ciclo de operación en un entorno productivo real.

Conclusion

El desarrollo del Portal News validó exitosamente una arquitectura de persistencia políglota, integrando MongoDB, Apache Cassandra y Redis para optimizar la lógica de negocio. Esta estrategia permitió especializar el manejo de datos: MongoDB agilizó la gestión de contenido flexible, Cassandra aseguró la disponibilidad en el registro de usuarios y Redis redujo la latencia de las transacciones de seguridad a tiempos mínimos. La separación de la API en Express y el cliente en SvelteKit facilitó la modularidad, permitiendo que la lógica de backend procesara eficientemente las reglas de negocio y la orquestación de datos sin afectar el rendimiento de la interfaz.

En cuanto al despliegue en la nube, se logró configurar correctamente la interconexión entre el backend en Railway y los servicios de base de datos distribuidos (Atlas y Astra DB). Sin embargo, la arquitectura distribuida enfrentó desafíos significativos relacionados con las políticas de seguridad modernas de los navegadores. Aunque la infraestructura de datos funcionó correctamente, la separación de dominios entre el Frontend (Vercel) y el Backend (Railway) generó restricciones con las cookies de terceros (Third-Party Cookies). Esto provocó que navegadores estrictos como Firefox revocaran las sesiones, impidiendo la persistencia del inicio de sesión en el entorno productivo, lo que evidencia la necesidad de estrategias adicionales como proxies inversos o dominios unificados para superar estas barreras de seguridad en arquitecturas *cross-site*

Link al repositorio:

https://github.com/JoseAdrianRodriguezGonzalez/Portal_news

Link a la pagina: https://portal-news-n5yd.vercel.app/